Class: TE **Subject: AISC** **Sem-V**

Experiment No: 3

Course Outcome: LO2 **Blooms Level: L3**

Aim: Implement Informed Search Technique for 15-Puzzle Problem using A* Algorithm

Abstract: The 15-Puzzle Problem is a well-known artificial intelligence search problem that involves arranging fifteen numbered tiles on a 4×4 grid by sliding them into an empty space until the goal configuration is achieved. This experiment focuses on implementing the A* (A-Star) informed search algorithm to efficiently solve the puzzle. A* combines the actual path cost from the initial state with a heuristic estimate of the remaining cost, enabling it to explore the most promising states first. The implementation demonstrates how heuristic-based search significantly reduces the number of explored states compared to uninformed search methods. This experiment helps in understanding state-space representation, heuristic functions, priority-based node expansion, and the practical application of informed search techniques in solving complex optimization and path-finding problems.

Sample Input and Output:
Case 1: Input:
Initial State:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 0 15
Goal State:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0

Output -
Step 0 (Initial State)
Move: None

1 2 3 4
5 6 7 8
9 10 11 12
13 14 0 15

g = 0
h = 1
f = 1
Step 1
Move: Right
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0

Class: TE **Subject: AISC** **Sem-V**

```
g = 1
h = 0
f = 1
Goal State Reached!
```

Class: TE **Subject: AISC** **Sem-V**

Theory: Introduction to the 15 Puzzle Problem

The **15-Puzzle Problem** is a classic problem in Artificial Intelligence used to demonstrate **state-space search** and **heuristic search techniques**. It consists of a **4 × 4 grid** containing **15 numbered tiles** and one **blank space**. The objective is to rearrange the tiles from an initial configuration to a predefined goal configuration by sliding one tile at a time into the blank space.

Unlike uninformed search algorithms, which explore states without guidance, **A* (A-Star)** is an **informed search algorithm** that uses heuristic information to efficiently determine the optimal path to the goal. It is widely used in pathfinding, robotics, game development, and puzzle-solving because it guarantees the shortest solution when an admissible heuristic is used.

The **A*** (A-Star) algorithm is an informed search technique that selects the most promising node by evaluating the following function:

$f(n)=g(n)+h(n)$

where:

- **g(n)** = Actual cost from the initial state to the current state.
- **h(n)** = Estimated cost (heuristic) from the current state to the goal state.
- **f(n)** = Total estimated cost of reaching the goal through the current node.

The node with the **lowest f(n)** value is expanded first.

Heuristic Function (Manhattan Distance)

The most commonly used heuristic for the 15-Puzzle is the Manhattan Distance.

For each tile,

$$\text{Distance}=[\text{Current Row}-\text{Goal Row}]+[\text{Current Column}-\text{Goal Column}]$$

The total heuristic value is calculated as:

$$h(n) = \sum_{i=1}^{15} ManhattanDistance(i)$$

The Manhattan Distance heuristic is **admissible**, meaning it never overestimates the actual cost. Therefore, A* always finds the **optimal solution** if one exists.

Working of A* Algorithm

Class: TE **Subject: AISC** **Sem-V**

Insert the initial state into the OPEN list.

1. Compute the values of **g(n)**, **h(n)**, and **f(n)**.
2. Select the state with the minimum **f(n)** value.
3. If the selected state is the goal state, terminate the search.
4. Otherwise, generate all valid successor states by moving the blank tile.
5. Calculate **g(n)**, **h(n)**, and **f(n)** for each successor.
6. Add unexplored successor states to the OPEN list and move the expanded state to the CLOSED list.
7. Repeat the process until the goal state is reached or no solution exists.

Algorithm: A* Search for the 15-Puzzle

Input: Initial puzzle configuration

Output: Optimal sequence of moves to reach the goal state

1. Start.
2. Define the goal state of the 15-puzzle.
3. Create an **OPEN list** (priority queue) and a **CLOSED list** (visited states).
4. Insert the initial state into the OPEN list.
5. Compute:
    - g(n)=0g(n) = 0g(n)=0
    - h(n)h(n)h(n) using Manhattan Distance.
    - f(n)=g(n)+h(n)f(n) = g(n) + h(n)f(n)=g(n)+h(n)
6. Repeat until the OPEN list is empty:
    - Remove the node having the smallest **f(n)** value.
    - If the node is the goal state:
        - Display the solution path.
        - Stop.
    - Otherwise:
        - Generate all possible successor states by moving the blank tile.
        - For each successor:
            - Compute its **g(n)** value.
            - Compute its **h(n)** using Manhattan Distance.
            - Compute **f(n) = g(n) + h(n)**.
            - If the state has not been explored, insert it into the OPEN list.
        - Add the expanded node to the CLOSED list.
7. If the OPEN list becomes empty, display **"No Solution Found."**
8. Stop.

Output: Solution Path

Move : None

1 2 3 4
5 6 7 8
9 10 11 12
13 0 14 15
g = 0
h = 2
f = 2

Move : Right
1 2 3 4
5 6 7 8
9 10 11 12
13 14 0 15
g = 1
h = 1
f = 2

Move : Right
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
g = 2
h = 0
f = 2

Goal State Reached!
Total Moves : 2
Search Completed Successfully.

Conclusion:

The 15-Puzzle Problem was successfully solved using the A* informed search algorithm. By employing the Manhattan Distance heuristic, the algorithm efficiently explored the search space and generated the optimal sequence of moves from the initial state to the goal state while minimizing the number of expanded nodes.

Code File:
https://colab.research.google.com/drive/1KlFG9YTv5aqmjYDS1lA3zRFVRzC-gffG?usp=sharing

Explanation File:
https://docs.google.com/document/d/1lODP_A95irG0huIlFlLl7z4eZIWPk0fC8brS0iWMDX8/edit?usp=sharing

Class: TE **Subject: AISC** **Sem-V**

Exercise 1: Implement water jug problem using A* algorithm

Abstract:

Solve the Water Jug Problem using A* Search Algorithm

Problem Statement:

A person has two water jugs of capacities 4 litres and 3 litres. Initially, both jugs are empty. Using the operations Fill, Empty, and Transfer Water, determine the sequence of steps required to obtain exactly 2 litres of water in the 4-litre jug using the A* Search Algorithm.

Initial State: (0, 0)

Goal State: (2, x)

where x can be any quantity of water in the 3-litre jug.

Exercise 2: Implement N Queen using A* algorithm

Abstract: The N-Queen Problem is a classical Artificial Intelligence problem that involves placing N queens on an N × N chessboard such that no two queens attack each other horizontally, vertically, or diagonally. This experiment implements the A* (A-Star) informed search algorithm to efficiently find a valid arrangement of queens by exploring the most promising states first. The A* algorithm evaluates each state using the function f(n) = g(n) + h(n), where g(n) represents the cost of reaching the current state and h(n) estimates the remaining cost based on the number of attacking queen pairs. By utilizing heuristic information, the algorithm significantly reduces the search space compared to uninformed search techniques and efficiently identifies a conflict-free solution. This implementation demonstrates the concepts of state-space representation, heuristic evaluation, priority-based node expansion, and informed search strategies, highlighting the effectiveness of the A* algorithm in solving combinatorial optimization problems in Artificial Intelligence.